Learning Fast Approximations of Sparse Coding (LISTA)

Caleb Marquart **Nafisa Maliha** **Russell Reid** **Victor Kwok**

1 Introduction

In this report we present a reproduction study of Gregor and LeCun (2010), "Learning Fast Approximations of Sparse Coding" [1] (hereafter referred to as "the paper"). We summarize the paper's core ideas and present a reproduction experiment of a central qualitative phenomenon of the paper.

2 Paper Summary

This paper addresses the computational obstacles that appear in sparse coding inference. Sparse coding is an unsupervised machine learning technique used for feature extraction and data representation. While it is a highly effective unsupervised method for feature extraction, traditional models like the Iterative Shrinkage and Thresholding Algorithm (ISTA) and Coordinate Descent (CoD) are computationally expensive because they require numerous iterations to converge for every new input. The authors of this paper propose another model called Learned ISTA (LISTA), which treats a fixed number of iterations of the ISTA algorithm as layers in a feed-forward neural network. By learning the filter matrices, mutual inhibition matrices, and shrinkage thresholds directly from the data instead of calculating them analytically, LISTA produces very accurate approximations of the optimal sparse code in a short amount of time. The paper shows that Learned ISTA produces faster and more optimized results than the more common methods like ISTA, enabling efficient real-time applications in computer vision and object recognition.

3 Core Ideas

How Sparse Coding Works

The key concept in sparse coding is linear reconstruction, described by the matrix equation below (1). An input signal X (with n dimensions) is modelled as a linear combination of basis vectors, which are stored as the columns of the dictionary matrix W_d. The vector Z represents the code containing the specific coefficients or weights applied to each basis vector to recreate the input. An overcomplete case, where the number of basis vectors (m) is greater than the input dimensions (n), allows more flexibility and better sparse representations, as having more basis vectors provides a large, redundant vocabulary of features and only a small fraction of these building blocks are required to accurately reconstruct the given input, X.

$$\underbrace{[x_1 \quad \cdots \quad x_i]}_{\mathbf{X}\in\mathbb{R}^{n\times i}} = \underbrace{[d_1 \quad \cdots \quad d_m]}_{\mathbf{W_d}\in\mathbb{R}^{n\times m}}\underbrace{[z_1 \quad \cdots \quad z_i]}_{\mathbf{Z}\in\mathbb{R}^{m\times i}} \tag{1}$$

The mathematical foundation of sparse coding relies on minimizing the specific energy function below (2). For a given input vector $X \in \mathbb{R}^n$ and an overcomplete dictionary of basis vectors $W_d \in \mathbb{R}^{n\times m}$ (where $m > n$), the goal is to find the optimal sparse code $Z^* \in \mathbb{R}^m$ that minimizes this energy function (2).

The first term, $\frac{1}{2}||X - W_d Z||_2^2$ in this equation is the 'Reconstruction Error', which applies an L_2 norm. The term ensures that the selected basis vectors from the dictionary, W_d, accurately recreate the original input image X. The second term, $\alpha||Z||_1$, is the 'Sparsity Penalty', which applies the L_1

norm. Unlike the L_2 norm, which penalizes large errors heavily but is tolerant of tiny non-zero noise, the L_1 norm penalizes all non-zero elements. This mathematically forces the optimizer to drive most elements in Z to exactly zero. α controls the strength of this sparsity constraint.

$$E_{W_d}(X, Z) = \frac{1}{2}||X - W_d Z||_2^2 + \alpha||Z||_1 \tag{2}$$

Traditional Sparse Coding Models: ISTA and CoD

ISTA (Iterative Shrinkage and Thresholding Algorithm) is one of the common methods in sparse coding where the optimization of the energy function relies on iterative solvers. ISTA updates the entire code vector simultaneously using the recursive rule in equation (3). It describes a single step in the classical iterative math algorithm used to find a sparse code.

$$Z(k+1) = h_\theta(W_e X + SZ(k)) \tag{3}$$

In this equation, the filter matrix W_e and the mutual inhibition matrix S are fixed and derived mathematically from the dictionary W_d. The filter matrix in equation (4) functions as the initial encoder, projecting the raw input signal into the dictionary's feature space to get a baseline prediction of the sparse code. The mutual inhibition matrix in equation (5) acts as a feedback mechanism that suppresses any redundant features to ensure that the final representation remains optimally sparse. Equation (6) is the shrinkage function, which acts as a soft-thresholding operator to enforce sparsity by setting the small values to exactly zero.

Filter Matrix:

$$W_e = \frac{1}{L} W_d^T \tag{4}$$

Mutual Inhibition Matrix:

$$S = I - \frac{1}{L} W_d^T W_d \tag{5}$$

Shrinkage Function:

$$[h_\theta(V)]_i = \text{sign}(V_i)(|V_i| - \theta_i)_+ \tag{6}$$

Coordinate Descent (CoD), unlike ISTA, updates only the single coordinate that will most rapidly decrease the energy function at each step. While extremely efficient and considered the "Fastest Iterative Algorithm" for sparse coding, CoD is still iterative and variable in computation time. In our reproduction experiment, CoD is utilized to generate the ground-truth optimal codes (Z^*) for training.

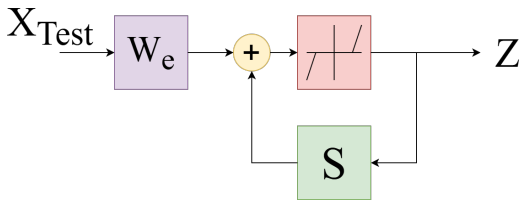

Figure 1: The diagram illustrates the traditional ISTA algorithm as an iterative loop. The input X is projected into the feature space by W_e. The signal then enters a feedback loop where the mutual inhibition matrix S adjusts the activations based on the previous state, and the shrinkage function h_θ sets the weak signals to zero to enforce sparsity. Because it is a closed loop, the data cycles through this process an unpredictable number of times until the output Z stops changing (convergence).

The Core of this Paper: Learned ISTA (LISTA)

This paper uses the concept of 'algorithm unrolling'. Traditional methods like ISTA iterate a fixed algorithm until convergence. LISTA, however, reuses this iterative optimization as a machine learning problem. By strictly limiting the algorithm to a finite number of iterations (T), the authors demonstrated that the ISTA loop can be "unrolled" into a T-layer, time-unfolded recurrent neural network. Instead of relying on the filter and mutual inhibition matrices derived from a fixed dictionary W_d, LISTA treats the matrices, along with the sparsity thresholds, as trainable parameters. This allows the network to learn highly optimized, data-specific ways to get the optimal sparse code. The LISTA process can be broken down into two key steps: forward propagation and backpropagation.

Forward Propagation: During forward propagation, the network takes a raw input signal X (like a vector image) and passes it through T identical layers to predict the sparse code. The process begins by projecting the input into the feature space using the learned filter matrix W_e, which creates the base prediction, B shown in equation (7)

$$B = W_e X \tag{7}$$

The shrinkage function, h_θ is applied to this base prediction B, which is fed into the unrolled layers. The network then updates its prediction at each layer k (where $k = 1, 2, \ldots, T$) using the recursive relationship in equation (8)

$$Z(k) = h_\theta(B + SZ(k-1)) \tag{8}$$

The mutual inhibition matrix, S and the shrinkage function, h_θ are two of the important parameters that are learned in this forward propagation step. After the signal passes through all T layers, the network outputs its final fast approximation of the sparse code.

Backpropagation: The network optimizes its learned parameters, the filter matrix W_e, the mutual inhibition matrix S, and the thresholds θ, by iteratively processing training samples. The gradients required to update these weights are computed using the standard back-propagation procedure. However, to calculate this error gradient during back-propagation, the network first requires a ground-truth target to compare its predictions against. Coordinate Descent (CoD) algorithm is run until full convergence to generate the optimal sparse codes (Z^*) for every sample in the training set. This is how the target required is generated prior to using it during backpropagation. The network is trained to minimize the Mean Squared Error (MSE) between its fast T-layer prediction and the CoD-generated target. The loss function for a batch of P training samples is defined by equation (9).

$$\mathcal{L}(W) = \frac{1}{P} \sum_{p=0}^{P-1} \frac{1}{2} ||Z^{*p} - f_e(W, X^p)||^2 \tag{9}$$

The architecture can be seen as a time-unfolded recurrent neural network to which the equivalent of back-propagation through time (BPTT) can be applied. During the backward pass, the gradient of the loss with respect to the network's prediction is computed at the final layer T. This error signal is then propagated backward through the unrolled layers. Because the matrices W_e and S are shared across all T layers, the gradients computed at each individual layer are combined to determine the total gradient for each parameter. Finally, the network updates its shared weights and thresholds simultaneously using a gradient descent optimizer. This paper used a standard Stochastic Gradient Descent (SGD) with the manual $1/j$ learning rate decay shown in equation (10).

$$W(j+1) = W(j) - \eta(j) \frac{\partial \mathcal{L}}{\partial W} \tag{10}$$

Through this combined forward and backward process, the LISTA network successfully achieves greater optimization and highly accurate sparse codes in a shorter time frame than the more traditional methods like ISTA.

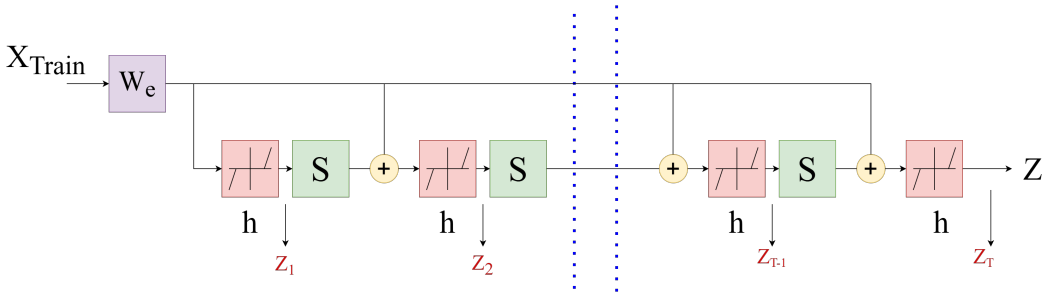

Figure 2: This diagram represents the main concept of 'Algorithm Unrolling.' The iterative loop from Diagram 1 is unrolled into a finite, feed-forward neural network with a fixed depth of T layers. Instead of the matrix S being a feedback loop, it is a shared weight matrix connecting each layer to the next. This architecture ensures a fixed computation time, as the signal only passes forward through the T stages, making it optimal for real-time inference and applications.

4 Reproduction Experiment

To verify the central claim of the paper, that a neural network trained to approximate sparse coding (LISTA) can achieve lower sparse code prediction error than FISTA in the same number of iterations, we reproduced the core experiment with slight changes to compare the code prediction error of LISTA against FISTA for varying iteration counts (T=1,3,7) and dictionary sizes (m=100,400).

Dataset
Our experiment uses the MNIST dataset: a dataset of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. We extracted 10x10 patches from the MNIST images, and flattened the patches to vectors of dimension n=100, matching the patch size used in the original paper. We generated a training set X_{train} of 50,000 patches, and a test set X_{test} of 5,000 patches. Each patch was normalized to zero mean and unit standard deviation; patches with a standard deviation below a minimum threshold were discarded, as they contain insufficient signal for meaningful sparse representation.

Dictionary Learning
In the dictionary learning step, we produce two dictionary matrices W_d for the $m = n = 100$ (complete) and $m = 4n = 400$ (4x overcomplete) cases. The paper assumes a fixed, pre-trained dictionary W_d which is "often learned by minimizing the average of $min_z E_{W_d}(X, Z)$ over a set of training samples using a stochastic gradient method." [1]

We trained W_d by first initializing it with random unit-norm columns. The training set is split into mini-batches of 256 image paches each, and on each batch we alternate between a CoD inference step to produce a rough Z^* estimate given the current W_d, and a gradient descent step on W_d using the Adam optimizer to minimize the reconstruction loss $||X - W_d Z^*||_2^2$. After each update each basis vector in W_d was re-normalized to unit norm. The process was repeated for 5 epochs.

Target Generation
After training the dictionaries, we run CoD to convergence (with tolerance of 1e-6 and a maximum of 500 iterations) on every training and test image patch to produce the ground-truth sparse code targets Z^*. These targets serve as the labels for LISTA training and are used in computing the prediction error for evaluating the performance of FISTA and LISTA.

FISTA Evaluation
To evaluate the performance of FISTA, we applied the FISTA algorithm to the test set X_{test} for 35 total iterations. The parameters W_e, S, and θ were computed analytically from the learned dictionary W_d. For each iteration the code prediction error $\frac{1}{2}\ ||Z^* - Z(k)||_2^2$, where k is the iteration number, was recorded. We performed this once for each dictionary size $m = 100$ and $m = 400$.

LISTA Training and Evaluation
The LISTA training step involves training 6 LISTA models in total, for dictionary sizes $m \in \{100, 400\}$, and unrolled depth $T \in \{1, 3, 7\}$. A total of 10 epochs were run for each model. The parameters W_e, S, and θ were initialized analytically from the trained W_d. In each epoch, we looped through each mini-batch of 256 patches from X_{train}, and performed a forward pass to compute Z, computed the prediction loss $\frac{1}{2}\|Z^* - \hat{Z}\|_2^2$, and backpropagated by taking a gradient descent step with the Adam optimizer, updating the parameters W_e, S, and θ.

The evaluation step involves evaluating the model on the test set X_{test} using the trained parameters W_e, S, and θ. For each model we run the unrolled LISTA algorithm for T iterations, recording the code prediction error $\frac{1}{2}\|Z^* - \hat{Z}\|_2^2$ at each iteration step.

To account for variability in training, each LISTA model was trained independently across 3 random seeds, with weight initialization and data shuffling varying between runs. The code prediction error reported for each LISTA model reflects the mean over the 3 seeds.

Results
The results for the code prediction error for each LISTA model, as well as for FISTA at T=1,3,7 were recorded in Table 1. Figure 3 and 4 shows the comparision of the code prediction error (in log scale) of the different LISTA models vs FISTA against number of iterations.

Table 1: Code prediction error ($\frac{1}{2}\|Z^* - Z(k)\|_2^2$) for LISTA and FISTA.

Method	$m = 100$	$m = 400$
LISTA $T = 1$	1.41	2.10
LISTA $T = 3$	0.90	1.57
LISTA $T = 7$	0.71	1.47
FISTA @ iter 1	14.20	13.00
FISTA @ iter 3	10.59	11.75
FISTA @ iter 7	6.34	9.90

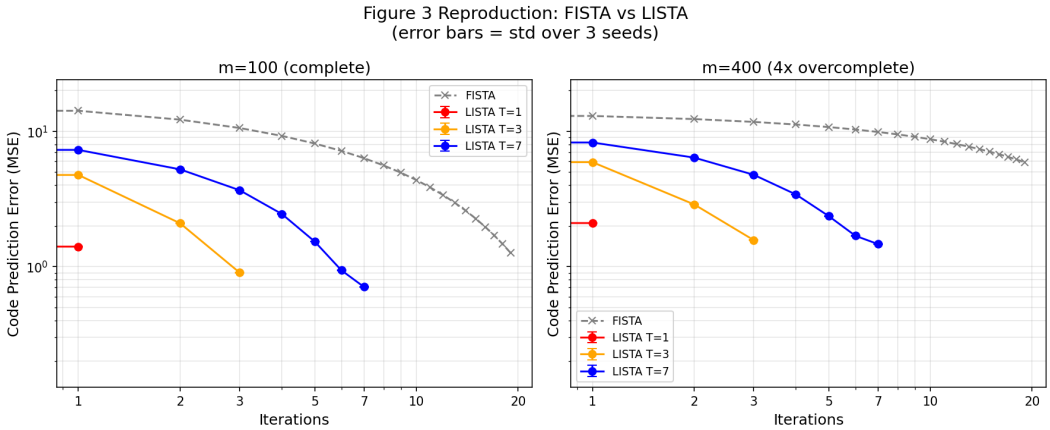

Figure 3: Reproduction of Figure 3 from Gregor & LeCun (2010). Code prediction error vs. number of iterations for FISTA and LISTA ($T = 1, 3, 7$) at $m = 100$ (left) and $m = 400$ (right). Both axes are on a logarithmic scale.

5 Critical Evaluation

Results of Our Experiment
Our reproduction experiment confirms the qualitative claim of the paper. We showed that LISTA achieves significantly lower prediction error compared to FISTA for the same number of iterations. As shown in our results, the Code Prediction Error (MSE) for LISTA lowers significantly within a few layers. For the complete dictionary case ($m = 100$), LISTA with just 1 layer ($T = 1$) achieved an error of 1.41. In comparison, it took FISTA 3 iterations to reach 10.59, and even at iteration 7, FISTA's error was 6.34. For the overcomplete dictionary case ($m = 400$), LISTA with $T = 3$ layers achieved an error of 1.57, drastically outperforming FISTA, which remained at an error of 9.90 even after 7 iterations. These results clearly show that LISTA easily outperforms FISTA in both complete and overcomplete cases and that using learned, data-specific parameters is a better choice to the simple iterative algorithms used by methods like ISTA and FISTA.

Discrepancy Between Our Results and the Original Paper
Our results match the qualitative findings of the paper, however, there exists some quantitative discrepancies. The numbers we obtained in our results were close but not identical to the paper's, as expected. For m=100, we obtained LISTA errors at T = 1,3,7 of 1.41, 0.90, and 0.71 respectively, compared to the paper's 1.50, 0.98, and 0.52. For m=400 we obtained errors of 2.10, 1.57, and 1.47 compared to the paper's 2.45, 2.12, and 1.62. For FISTA at iteration 1, we obtained errors of 14.20

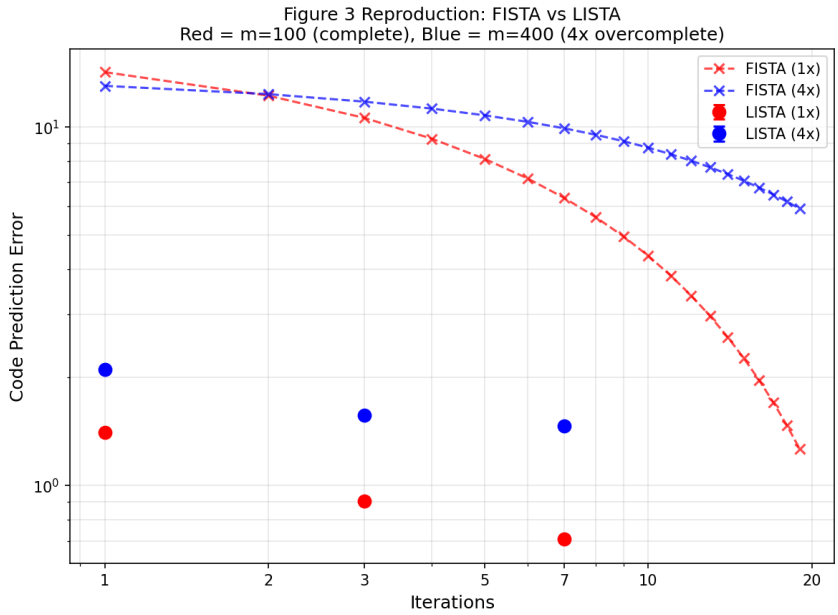

Figure 4: Reproduction of figure 3 from the paper, comparing all models from our experiments. The LISTA errors are shown with only the error of the final iteration.

and 13.00 for m = 100, 400 compared to the paper's 21 and 22. Overall we observe lower error in our results compared to the paper's results.

The most likely cause of these discrepancies is the different datasets used in our experiment compared to the paper's. The original paper used 10×10 patches from the Berkeley natural image dataset, which contain diverse, high-frequency textures. In our experiment we used 10×10 patches from the MNIST dataset of handwritten digits which is simpler, containing flat regions and the digit strokes. Since LISTA's advantage comes from learning data-specific inhibition patterns, it may produce lower error at lower iterations for the simpler MNIST dataset.

Another possible source of discrepancy is the different optimizer we used in our LISTA backpropagation step. The paper used online SGD with a $1/j$ learning rate decay, processing one sample at a time. In our experiment, we used mini-batch Adam with StepLR decay for a fixed 10 epochs, following the training loop structure from notebook 16 of the ECE447 course notes [2]. Adam provided more stable convergence than SGD in our setup, and mini-batching reduces the number of gradient steps per epoch from all 50,000 to approximately 196, speeding up training time.

Strengths and Limitations of our Experiment

Our experiment reproduced the paper's core structure and we were able to confirm the qualitative claim of the paper. We used the same dictionary sizes (m=100, 400), unrolled depths (T=1,3,7), evaluation metric, and CoD-generated targets for LISTA training. The correct qualitative behavior was reproduced across all conditions: prediction error decreased with increasing depth (T=7 > T=3 > T=1), and LISTA significantly outperformed FISTA at every iteration count and both dictionary sizes.

A main limitation of our experiment is that we did not explore LCoD (learned CoD). The original paper presents LCoD alongside LISTA as a second learned approximation method, applying the same unrolling concept to Coordinate Descent rather than ISTA. Reproducing LCoD would have provided a more complete comparison between learned and non-learned sparse coding methods, but we decided to just focus on LISTA.

Additionally, our experiment used MNIST patches rather than the Berkeley natural image dataset used in the original paper. Since natural images contain more diverse, high-frequency textures, testing on them would have provided a more direct and rigorous comparison to the original paper's results.

6 Reflections

Caleb Marquart:

From our project, working on the LISTA reconstruction, I was able to learn much more about sparse coding since it was the foundation of the paper and we had to research and understand the specifics of how it works including what each of the iterative algorithms does to achieve sparse codes. By doing this, I understood how the shrinkage function works to minimize the energy function to get the sparse code and how each of the parameters each serve a purpose to generate the sparse code. The main thing that this taught me about sparse coding is that the parameters (like the dictionary and mutual inhibition matrix) that are normally fixed can be learned using a neural network by updating the parameters during training of the model. This kind of felt like magic at first, but after having to write the Python code to do this process, it made a lot of sense and revealed that model training is mostly just matrix math and clever algorithms. I enjoyed this project and it was very valuable to have to read an academic paper and struggle through adequately understanding the experiment since recreation is not possible without a certain degree of understanding.

From the other group presentations, the theme I found in most of them is that each paper sought to improve a current learning strategy by addressing failures, speeding up networks, or dealing with limited data. What this indicates to me is that while neural networks are a powerful tool, there are usually limitations in speed or accuracy that need to be solved to make them even better. I found this specifically in the Mixup presentation, which showed how large models trained with Empirical Risk Minimization tend to memorize training data rather than generalize from it. The solution was to blend pairs of training samples and their labels together which was surprisingly simple but effective, even if it introduced its own challenges. It was interesting to see how each paper identified a different bottleneck, whether that was the training data, the model architecture, or the algorithm itself like ISTA in our LISTA paper, and then took a creative approach to solving it.

Nafisa Maliha:

What I learned from our project: Working on the LISTA project helped me get a better understanding of the relationship between deep learning and mathematics. Before this, I underestimated how connected math and machine learning actually is. Researching various concepts to better understand this paper actually clarified come of my confusions regarding neural networks. I had gaps in my knowledge of how the inputs mapped to the outputs in neural networks and my knowledge of sparse coding before this was negligible. However, since sparse coding is the base of this paper, I took the time to actually learn about it in detail and now I can explain what sparse coding is even mathematically and how it can be optimized using a number of different methods like ISTA, CoD and mainly in this paper, LISTA. I also understand the mathematical behavior of these processes and how we need to minimize the energy function for sparse coding to increase efficiency. I found myself enjoying learning how the shrinkage function, the mutual inhibition and filter matrices worked and the purpose they served, because when I first saw some of the equations in the paper, I found them quite intimidating. However, they turned out to be less complex than I expected. When I understood these terms, I realized how sparsity was actually achieved in this process. I also think studying this paper and actually running our own experiment which also produced similar results, made me realize how neural networks using learned parameters are almost always superior to the simple iterative algorithm when it comes to efficiency and accuracy.

What I learned from other group projects: I found the 'superposition disentanglement of neural networks' paper to be educational because, before watching this presentation, I had never thought of superposition in any relation to neural networks. So, learning that in superposition, networks map more features than they have neurons by overlapping them was very interesting. I also enjoyed connecting their use of Sparse Autoencoders to separate their features into a more sparse space for better alignment as I found that to be a bit similar to how we used filters to ensure our optimal Z matrix was sparse. Another group presentation that focused on the 'Mixup' paper demonstrated the flaws of standard Empirical Risk Minimization (ERM). I learned that large models tend to overfit or memorize data instead of generalizing. Listening to how they attempted to minimize this issue was interesting, although I did not grasp the math and algorithm involved clearly. I still found the knowledge beneficial as I would not have learned as much about neural networks and machine learning as I did with this project.

Russell Reid:

Working on this project gave me a much deeper appreciation for the intersection of optimization theory and sparse coding. Coming in, I had a working understanding of neural networks, but I had never considered that a classical iterative algorithm could itself be transformed into a learnable architecture. The concept of algorithm unrolling was the central insight that changed how I think about the relationship between optimization and learning. Rather than treating ISTA as a black-box solver, the paper takes each iteration as a layer with trainable parameters, and that unlocks a fundamentally more efficient solution. Seeing this idea go from theory to working code made it genuinely stick.

Contributing across different parts of the project, from preprocessing and dictionary learning to the LISTA implementation and writeup, gave me a broad view of how all the pieces fit together. I came to appreciate how much the quality of the dictionary Wd and the CoD-generated targets Z* mattered for LISTA training. A weakness in either upstream step would propagate directly into the final results. This end-to-end perspective also helped clarify why the quantitative discrepancies between our results and the original paper were not a failure of our implementation, but a natural consequence of using a different dataset and a more practical optimizer.

From other group presentations, I found the talk on Batch Normalization particularly relevant to our own work. Learning that deep networks struggle because the input distribution each layer sees keeps shifting as earlier layers update connected directly to the training challenges we faced. The idea of normalizing layer inputs over mini-batches to stabilize training and allow higher learning rates was interesting to see in contrast to our own use of mini-batching, which we adopted primarily to make LISTA training computationally feasible rather than for stability. Seeing how the same technique can serve different purposes depending on the context was a useful reminder that many tools in deep learning are more general than they first appear.

Victor Kwok:

The main thing I learned from the project is the skill of dissecting a dense academic paper to understanding. On the first pass I could only understand some of the high level details and it was difficult to grasp the mathematical concepts. Deeper understanding was gained only after multiple passes. Recreating the experiment without a deep level of understanding of the entire paper meant that I had to be comfortable with partial understanding. From the project I gained insight into concepts like sparse coding, and the method of optimizing a sparse code through iterative algorithms. By unrolling the iterative algorithm into a fixed number of steps and performing forward and back propagation on these unrolled steps, the algorithm can be used for training a model to learn a data-specific approximation, which performs much better on new data compared to the original iterative algorithm. The project strengthened my understanding of neural networks in that the structure of a neural network is not limited to classification or regression, but can also be derived from an iterative optimizer.

What I learned from other groups: From the presentation on "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift", I learned that training deep networks is complicated by the fact that the input distribution each layer sees keeps shifting as earlier layers update, which slows down training. Batch normalization addresses this by normalizing layer inputs using statistics computed over the current mini-batch, stabilizing these distributions and allowing the network to train faster and with higher learning rates. This connected to our project since we also applied mini-batching to speed up the training of our model.

7 References

[1] Gregor, K. & LeCun, Y. (2010) Learning Fast Approximations of Sparse Coding. *Proceedings of the 27th International Conference on Machine Learning (ICML).*

[2] B. Tolooshams, "Notebook 16: Sparse Autoencoders," ECE 447, University of Alberta, 2026. Unpublished course notes.